LEXICON

NOSQL & MONGODB

Meet MongoDB

Documents, collections and your first Python queries

LEXICON

Today

- After today you can:
  - Explain what NoSQL and MongoDB are, and how they differ from SQL
  - Know the words document, collection, field and _id
  - Look at your data from Python with print and pprint
  - Connect to MongoDB from Python with pymongo
  - Insert and find documents

LEXICON

From tables to documents

- Last two weeks: SQL. Tables, rows, columns, JOINs
- Every row in a table has exactly the same columns
- This week: NoSQL, a different way to store data
- NoSQL means 'Not only SQL'. It's not 'anti SQL'

LEXICON

Why do people use NoSQL?

Problem	NoSQL answer
The data changes shape often	Flexible schema: no CREATE TABLE needed
The app works with objects, not tables	Data is stored like objects (JSON)
Everything is spread over many tables	Store things that belong together in one place
Huge amounts of data	Built to spread over many servers

LEXICON

MongoDB

- The most popular NoSQL database
- A document database: data is stored as documents
- Free: MongoDB Community Server runs on your own computer
- This week we talk to MongoDB from Python, with pymongo

LEXICON

A document

- Looks just like a Python dictionary (or JSON)
- Pairs of field: value

```
{
  "_id": 1,
  "name": "Hoodie Black",
  "category": "Clothing",
  "price": 599,
  "tags": ["winter", "cotton", "bestseller"],
  "details": {"color": "black", "material": "cotton"}
}
```

LEXICON

SQL words and MongoDB words

SQL	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary key	_id
JOIN	Embedding, or $lookup
SQL query	A filter (a Python dictionary)

LEXICON

Flexible schema

- Documents in the same collection can have different fields
- Good: easy to add new kinds of data
- Bad: nobody stops a typo like 'prise' instead of 'price'

```
{"_id": 2, "name": "T-shirt White", "price": 249}
{"_id": 4, "name": "Sneakers Classic", "price": 1199,
 "sizes": [39, 40, 41, 42, 43, 44]}
```

LEXICON

_id: every document's ID

- Every document has an _id field, like a primary key
- It must be unique in the collection
- Don't give one? MongoDB makes an ObjectId for you
- Our webshop uses 1, 2, 3... so it's easy to compare with SQL

```
_id: ObjectId('6720f1a2b3c4d5e6f7a8b9c0')
```

LEXICON

Data types

Type	Example	Python
String	"Uppsala"	str
Int / Double	599 / 4.5	int / float
Boolean	true	True
Date	2026-01-05	datetime
Array	["winter", "cotton"]	list
Object	{"color": "black"}	dict
Null	null	None
ObjectId	ObjectId('6720...')	ObjectId

LEXICON

A whole receipt in one document

- In SQL this was 3 tables: orders, order_items, products
- Here: one document, no JOIN needed

```
{
  "_id": 1, "customer_id": 1, "status": "delivered",
  "order_date": ISODate("2026-01-05"),
  "items": [
    {"name": "Hoodie Black", "quantity": 1, "unit_price": 599},
    {"name": "Cap Logo", "quantity": 2, "unit_price": 199}
  ]
}
```

LEXICON

Our tools this week

- Address of the server on your computer: mongodb://localhost:27017

Tool	What it is	Like…
MongoDB Server	The database. Runs in the background	The kitchen
pymongo	The Python library to talk to MongoDB	sqlite3 (the waiter)
VS Code	Where we write and run our Python	Same as always

LEXICON

pymongo in four lines

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")  # 1. connect
db = client["webshop"]                              # 2. pick a database
product = db.products.find_one()                    # 3. ask a question
print(product)                                      # 4. use the answer
```

LEXICON

Find and insert

```
db.customers.find_one({"_id": 1})           # one document (or None)
db.customers.find({"city": "Uppsala"})     # many documents: loop over them
db.customers.count_documents({})             # how many?

db.customers.insert_one({"first_name": "Noah", "city": "Lund"})
db.products.insert_many([{...}, {...}])
```

LEXICON

Filters are dictionaries

- In Python, field names need quotes
- A spelling mistake gives 0 results, not an error!

MongoDB filter	SQL
{}	(no WHERE: everything)
{"city": "Uppsala"}	WHERE city = 'Uppsala'
{"category": "Shoes", "stock": 0}	WHERE category = 'Shoes' AND stock = 0

LEXICON

Remember

- MongoDB stores documents in collections
- A document is like a Python dictionary, with lists and objects inside
- Every document has a unique _id
- pymongo = talk to MongoDB from Python
- Filters are dictionaries: {"city": "Uppsala"}

LEXICON

This afternoon: lab

Open the exercise PDF. Work at your own pace, and ask for help when you get stuck.

LEXICON

Coming up tomorrow

Tomorrow: smarter questions (greater than, OR, arrays, sorting) and how to change and delete documents.